

Your grade: 100%

Your latest: 100% • Your highest: 100% • To pass you need at least 80%. We keep your highest score.

Next item →

1.

What is the flow of a computation in the Von Neumann architecture? (video 1)

1 / 1 point

☐

All program instructions are stored in memory; all data are stored in the registers in the CPU; instructions are moved one-by-one into the control unit; data in the registers are operated on by the arithmetic logic unit

☒

All program instructions and data are stored in memory; instructions are moved into the control unit within the CPU; data are moved into the registers in the CPU and operated on by the arithmetic logic unit

☐

All program instructions and data are stored in memory; instructions are moved into the arithmetic logic unit in the CPU; data are moved into the control unit and operated on by the instructions through the registers

☐

All program instructions are stored on the control unit within the CPU; all data are stored in memory, moved into registers in the CPU, and operated on by the arithmetic logic unit

Nice work

Correct. Main memory has enough space to store both the program instructions and data, and the CPU has the components necessary to perform operations on the data in the registers with the arithmetic logic unit, according to the instructions in the control unit
2.

Which of the following is least useful when optimizing a loop for vectorization? (video 2)

1 / 1 point

☐

Eliminating pointer aliasing when writing in C or C++ (referencing the same location

☒

Using branching or conditionals within the loop

☐

Aligning data on register boundaries

☐

Including directives for vectorization in your code and passing vectorization flags to the compiler itself when compiling

Nice work

Correct. Often, doing this will prevent the compiler from vectorizing your code at all
3.

What purpose does data alignment serve? (video 3)

1 / 1 point

☐

To facilitate cache loading

☐

To avoid memory leaks

☐

To align data used in a program with the CPU's registers, allowing for efficiency in SIMD

☒

Both to align data used in a program with the CPU's registers, and to facilitate cache loading

Nice work

Correct. Data alignment serves both purposes. When data are aligned with register boundaries, SIMD can operate in a more orderly way, and fewer cache misses occur, which increases efficiency.
4.

Which type of parallelism utilizes a single instruction to perform operations on multiple data elements at once?

1 / 1 point

☐

Task parallelism

☒

SIMD

☐

MIMD

☐

Bit-level parallelism

Nice work

Correct. This performs the same operation on multiple data elements simultaneously.
5.

What is loop unrolling in the context of vectorization?

1 / 1 point

☐

Increasing the number of loop iterations

☒

The process of transforming individual loop operations into parallel vector operations

☐

Eliminating conditional branches in a loop

☐

Aligning loop data to register boundaries

Nice work

Correct. This is the definition of loop unrolling for vectorization
6.

Which of these is NOT a requirement for efficient vectorization of a loop?

1 / 1 point

☐

The loop trip count must be known before the loop starts

☐

The loop cannot contain function calls

☐

The loop data must be aligned

☒

The loop can contain branching/conditional statements

Nice work

Incorrect, branching inhibits vectorization.
7.

What is the purpose of the posix_memalign function in C?

1 / 1 point

☐

To create a vector data structure

☒

To allocate memory aligned for efficient vectorization

☐

To enable OpenMP SIMD directives

☐

To unroll loops automatically

Nice work

Correct, this allocates memory aligned to specific boundaries for vectorization.
8.

In C/C++, how would you use the _mm_malloc function to allocate memory aligned to a 32-byte boundary for a float array of size 1024?

1 / 1 point

☐

float* arr = _mm_malloc(sizeof(float)*1024, 32);

☒

float* arr = (float*)_mm_malloc(1024*sizeof(float), 32);

☐

float* arr = _mm_malloc(1024, sizeof(float)*32);

☐

float* arr = (float*)_mm_malloc(32, 1024*sizeof(float));

Nice work

Correct usage - _mm_malloc(size, alignment).
9.

What does the OpenMP 'reduction' clause tell the compiler about a loop?

1 / 1 point

☐

That the loop can be safely vectorized

☒

That the loop performs a reduction operation that can be vectorized

☐

That the loop data is aligned to specific boundaries

☐

That the loop does not require dependencies or aliasing

Nice work

Correct. The 'reduction' clause identifies a reduction operation to vectorize.
10.

Which optimization technique involves moving code that does not change within a loop outside of the loop?

1 / 1 point

☐

Loop fusion

☐

Loop fission

☒

Loop hoisting

☐

Loop interchange

Nice work

Correct, hoisting moves loop-invariant code outside the loop.